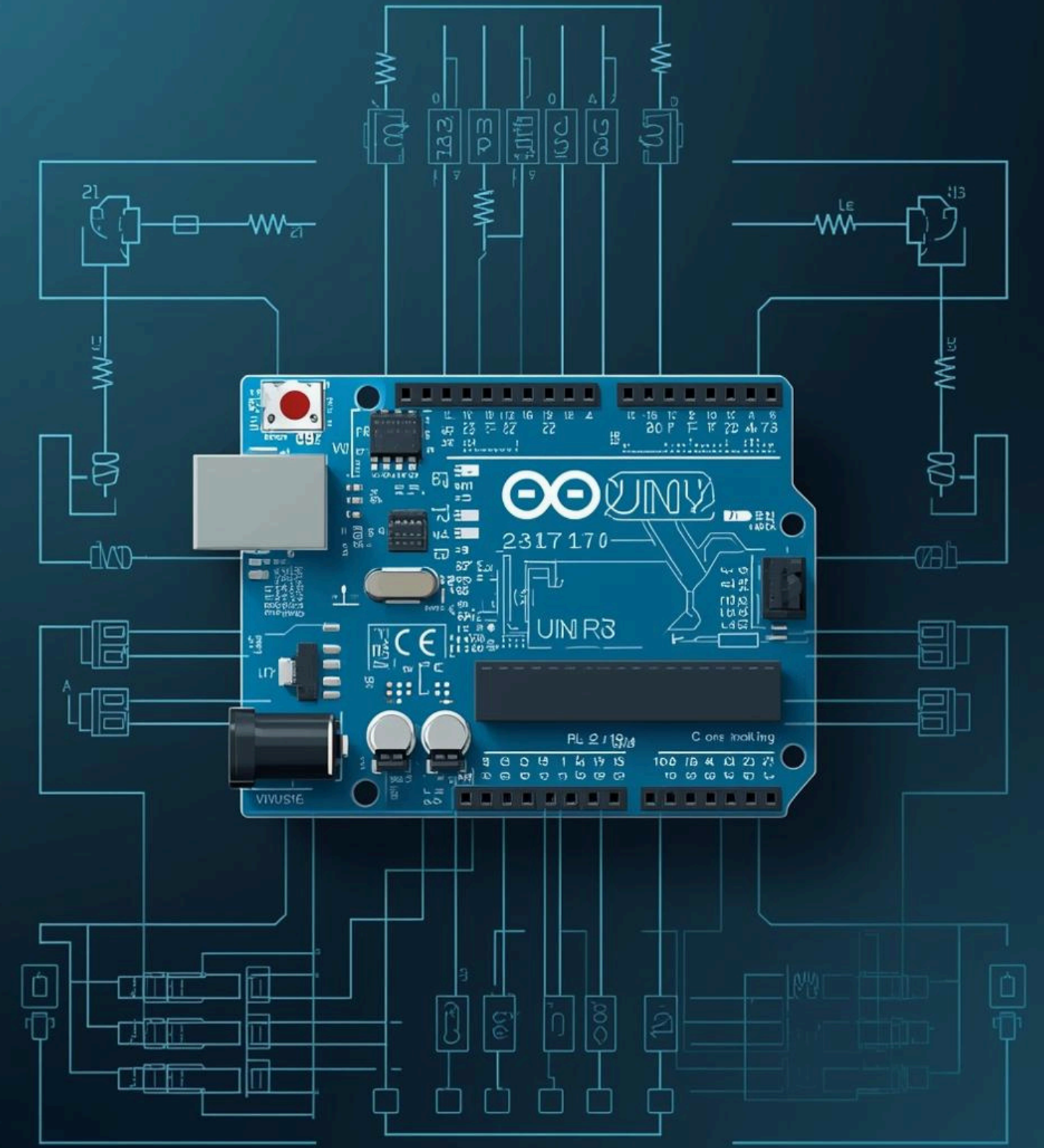


Side-Channel Attacks

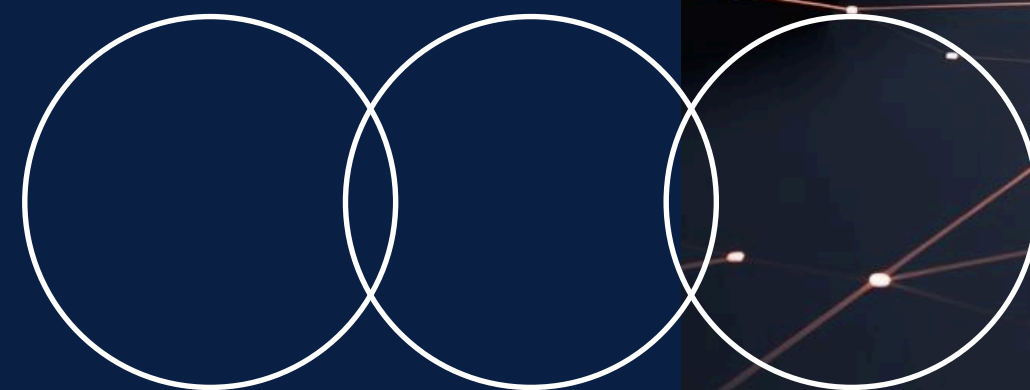
Presented by Vincent Liu



WHAT?

WHY?

HOW?



What Is a Hack?



Why Side-Channel Attacks?

Main Idea

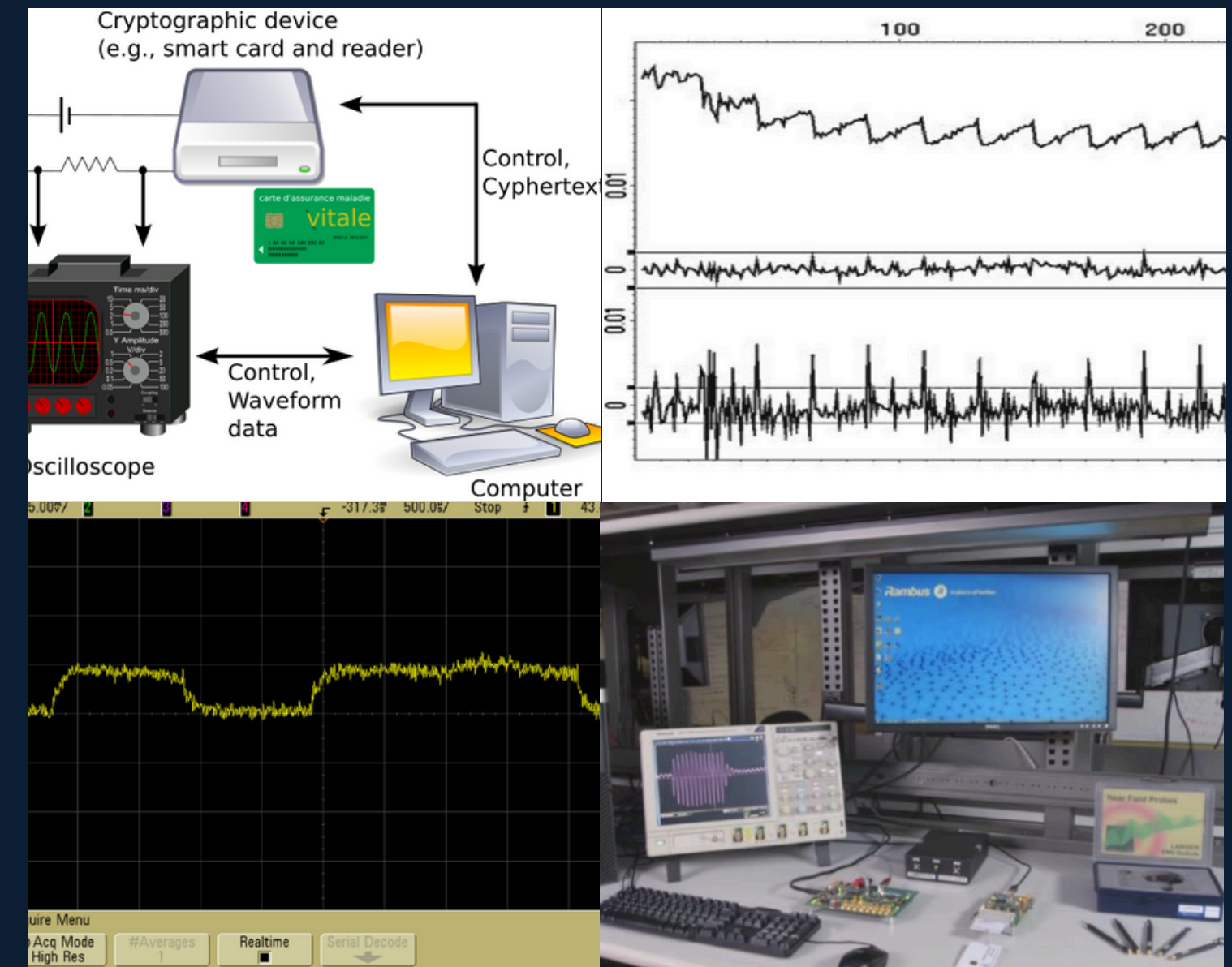
Even when cryptography is mathematically strong, the implementation can leak information.

Key Points

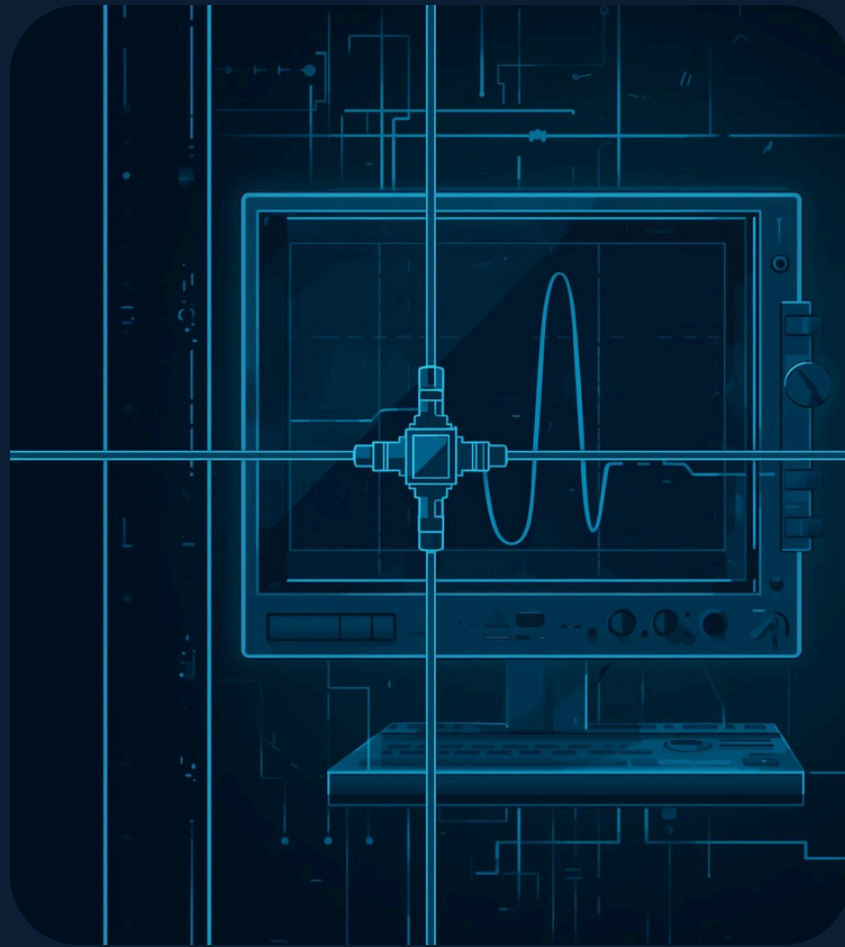
- Not attacking the algorithm
- Attackers use physical signals
- Low-cost and practical
- Real-world systems are vulnerable

Simple Explanation

Observing how the device behaves physically while encrypting, and using these leakages to recover the secret key.”



How Power Analysis Works?



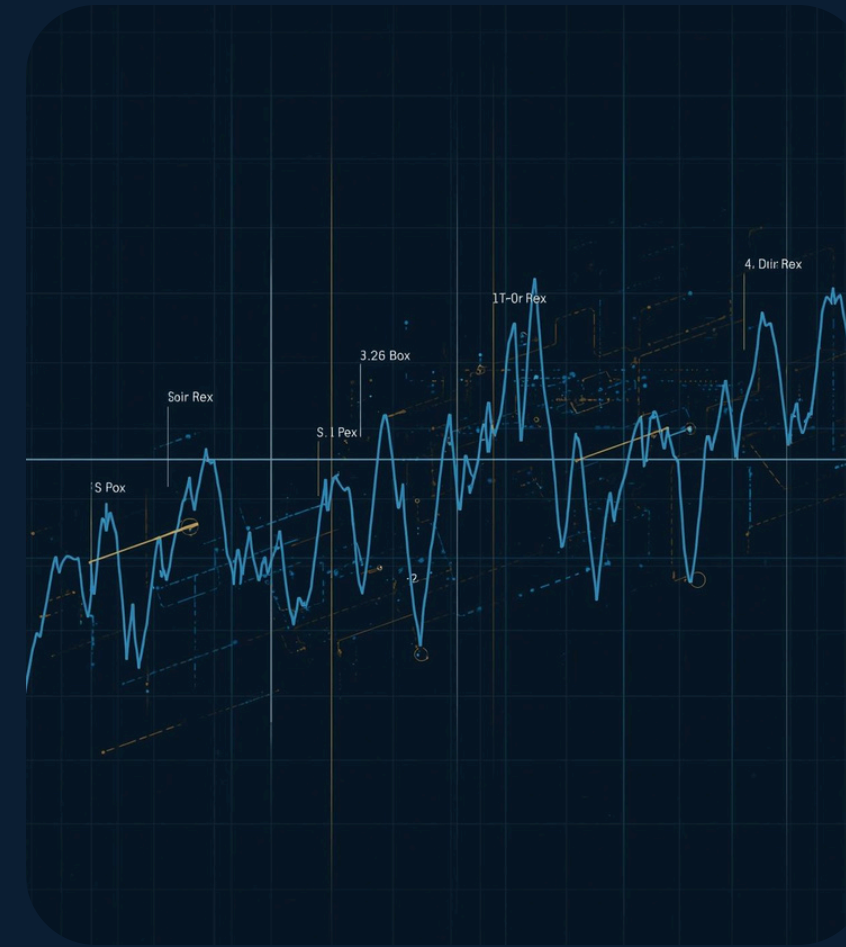
Measure Voltage

Shunt resistor measures voltage drops in power lines.



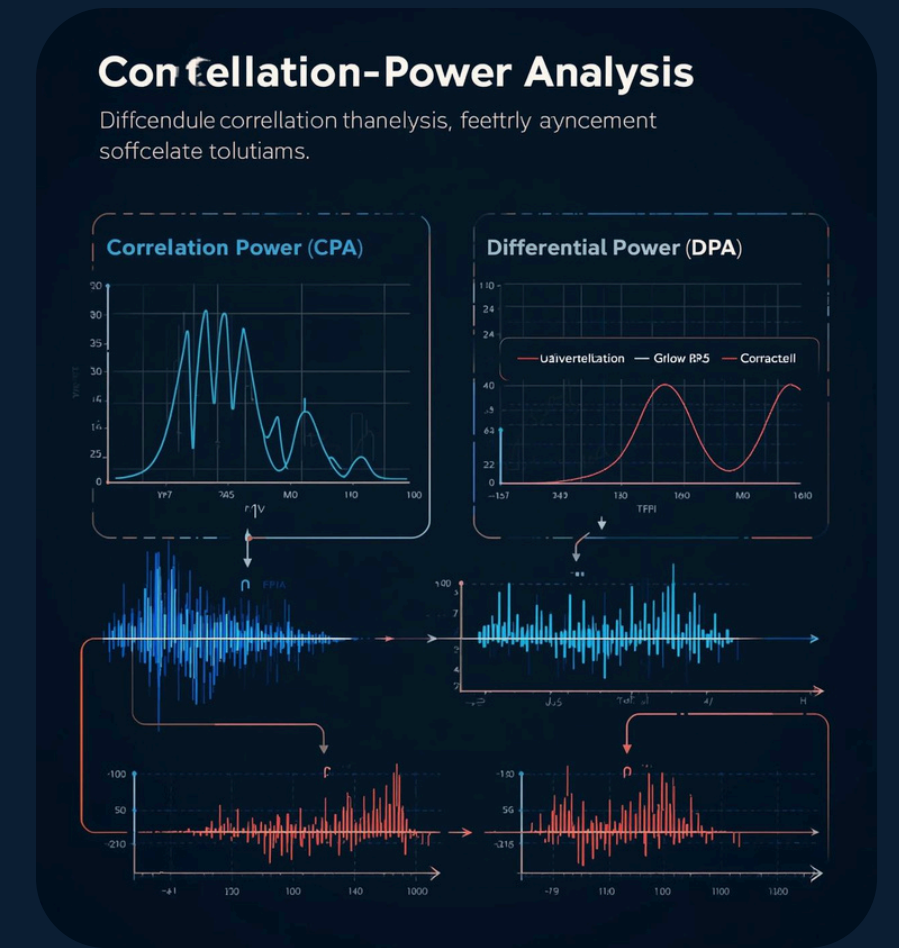
Collect Traces

Power traces are collected during cryptographic operations.



Identify Leakage

Waveform identifies critical leakage locations effectively.



Run Analysis

CPA/DPA correlates predictions against actual measurements.

Project Objectives

01 ATTACKER INSIGHT

I am not teaching how to hack — instead learning how attacks work so we can defend, harden, and protect systems.

02 ETHICAL STUDY

The goal is to find problems, not to cause harm.
This lets us test devices responsibly while keeping our work focused on defense, not hacking.

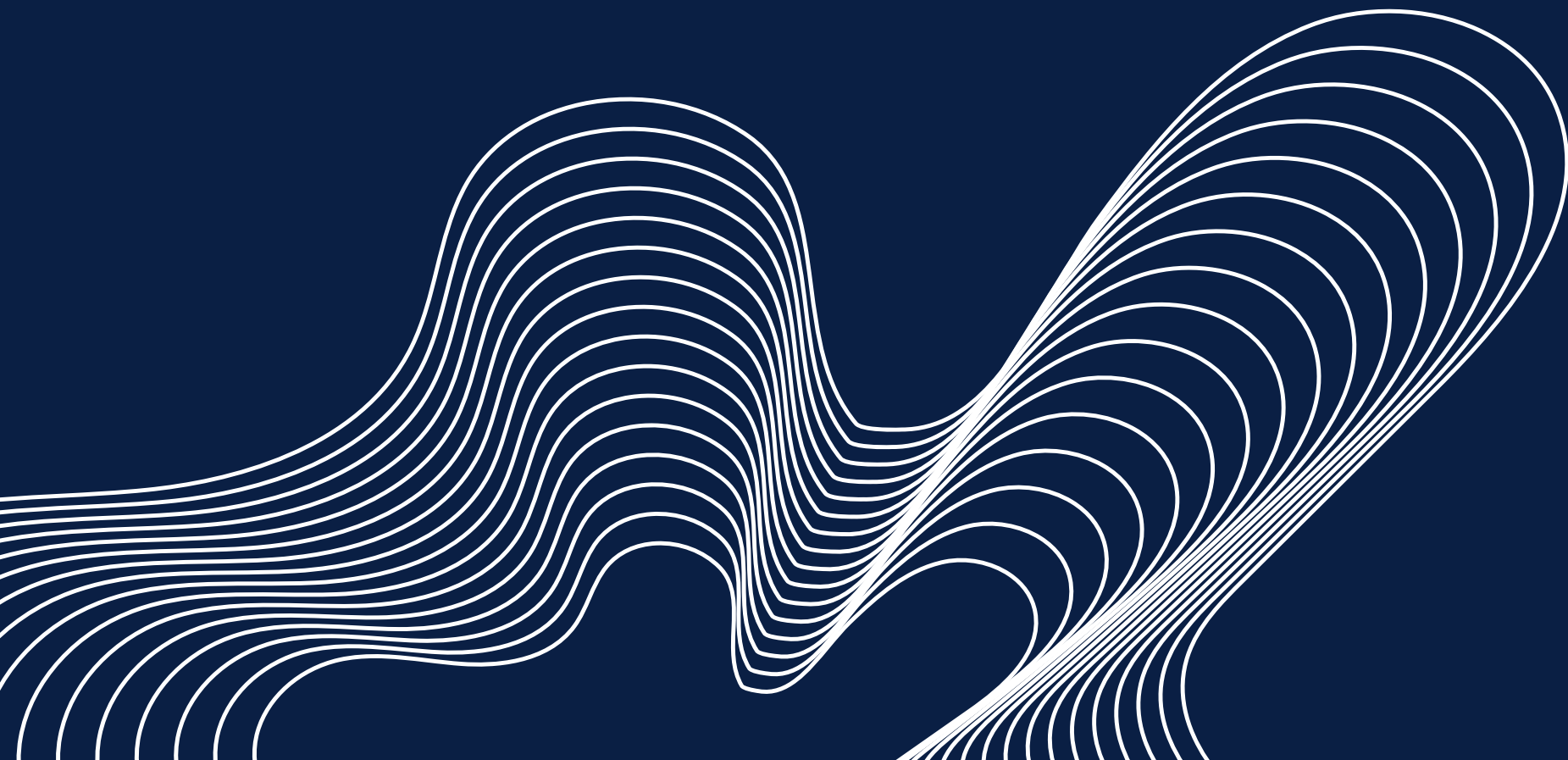
03 POWER ANALYSIS

Different small power changes — these can reveal secrets.
By studying these changes, we learn how much information a device accidentally leaks.

04 EARLY IDENTIFICATION

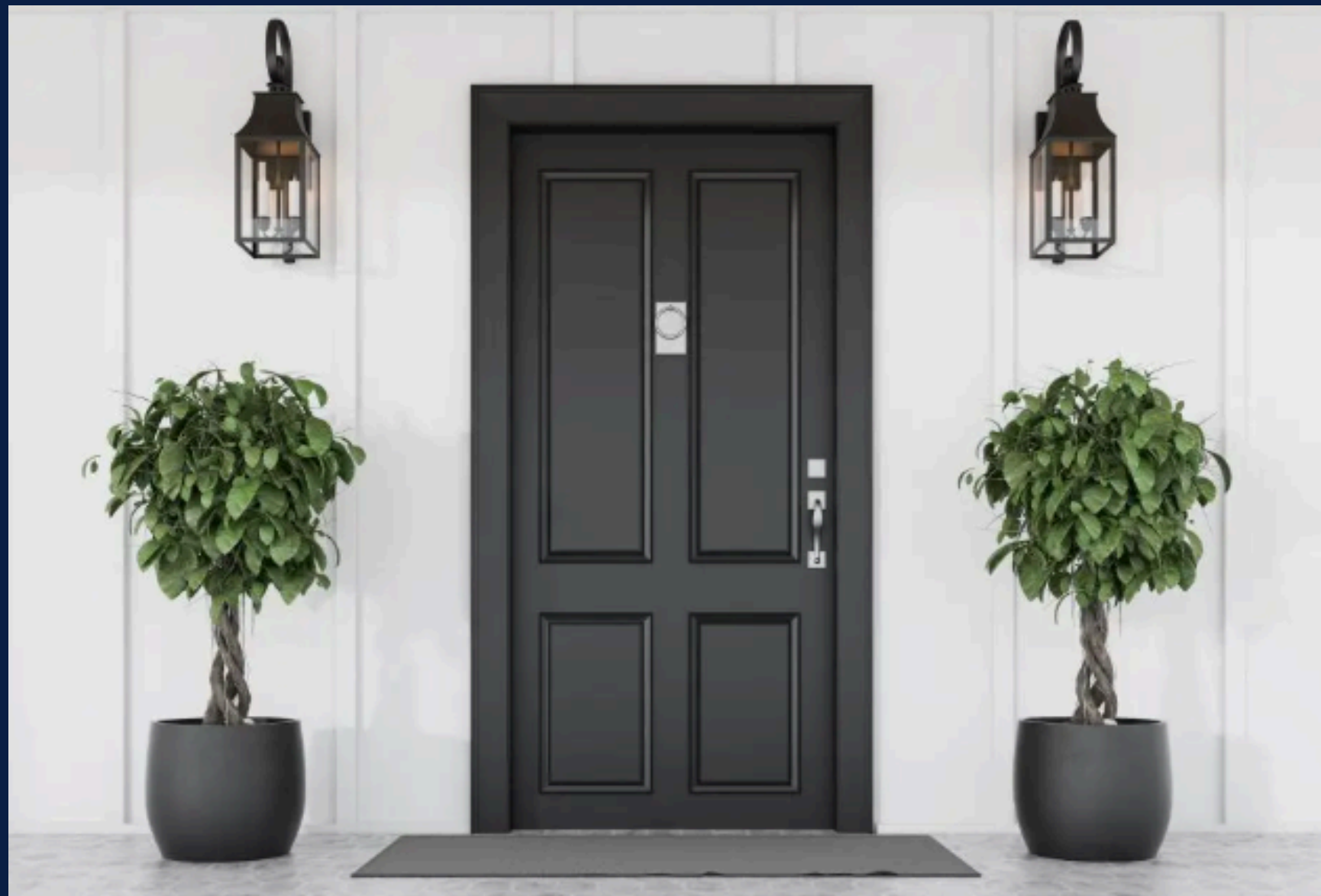
This helps designers build stronger and safer systems before they are used in the real world.

STAR Method



Situation: The Locked Door

**TODAY, I AM A THIEF TRYING TO BREAK INTO VINCENT'S HOUSE.
'UNFORTUNATELY, AN ELECTRONIC COMBINATION LOCK STOPS ME!**



Two Ways to Enter the Locked Door

OPTION 1: BREAK THE DOOR OPEN

Fast but noisy
Neighbors will notice
Police may arrive

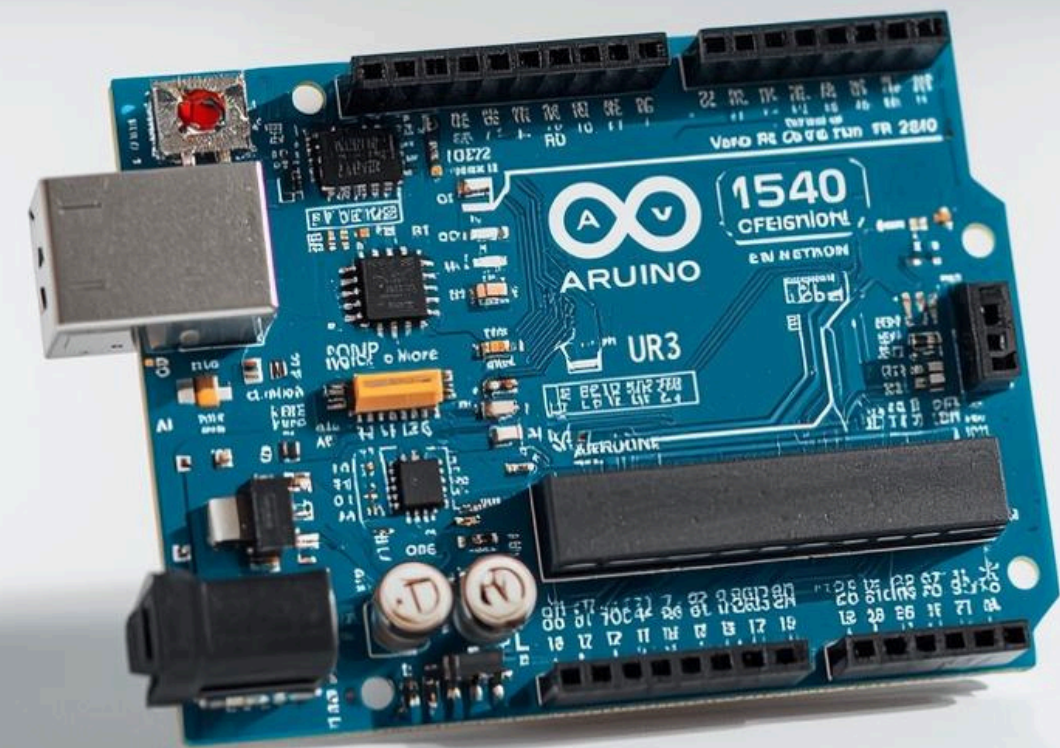


Option 2: Smart Way Use a Side-Channel Attack

Silent and stealthy
No damage
Extract passkey from digital lock



Task: Hack the Digital Passkey Door




```
sketch_nov30c.ino
10 #include <avr/io.h>
11 #include <util/delay.h>
12
13 // ===== AES S-box =====
14 static const uint8_t sbox[256] PROGMEM = {
15     0x63,0x7c,0x77,0x7b,0xf2,0x6b,0x6f,0xc5,0x30,0x01,0x67,0x2b,0xfc,0xd7,0xab,0x76,
16     0xca,0x82,0xc9,0x7d,0xfa,0x59,0x47,0xf0,0xad,0xd4,0xa2,0xaf,0x9c,0xa4,0x72,0xc0,
17     0xb7,0xfd,0x93,0x26,0x36,0x3f,0xf7,0xcc,0x34,0xa5,0xe5,0xf1,0xd8,0x31,0x15,
18     0x04,0xc7,0x23,0xc3,0x18,0x96,0x05,0x9a,0x07,0x12,0x80,0xe2,0xeb,0x27,0xb2,0x75,
19     0x09,0x83,0x2c,0x1a,0x1b,0x6e,0x5a,0xa0,0x52,0x3b,0xd6,0xb3,0x29,0xe3,0x2f,0x84,
20     0x53,0xd1,0x00,0xed,0x20,0xfc,0xb1,0x5b,0x6a,0xcb,0xbe,0x39,0x4a,0x4c,0x58,0xcf,
21     0xd0,0xef,0xaa,0xfb,0x43,0x4d,0x33,0x85,0x45,0xf9,0x02,0x7f,0x50,0x3c,0x9f,0xa8,
22     0x51,0xa3,0x40,0x8f,0x92,0x9d,0x38,0xf5,0xbc,0xb6,0xda,0x21,0x10,0xff,0xf3,0xd2,
23     0xcd,0x0c,0x13,0xec,0x5f,0x97,0x44,0x17,0xc4,0xa7,0x7e,0x3d,0x64,0x5d,0x19,0x73,
24     0x60,0x81,0x4f,0xdc,0x22,0x2a,0x90,0x88,0x46,0xee,0xb8,0x14,0xde,0x5e,0x0b,0xdb,
25     0xe0,0x32,0x3a,0x0a,0x49,0x06,0x24,0x5c,0xc2,0xd3,0xac,0x62,0x91,0x95,0xe4,0x79,
26     0xe7,0xc8,0x37,0x6d,0x8d,0xd5,0x4c,0xa9,0x5c,0x56,0xf4,0xea,0x65,0x7a,0xae,0x08,
27     0xba,0x78,0x25,0x1c,0xa6,0xb4,0xc6,0xe8,0xdd,0x74,0x1f,0x4b,0xbd,0x8b,0x8a,
28     0x70,0x3e,0xb5,0x66,0x48,0x03,0xf6,0xde,0x61,0x35,0x57,0xb9,0x86,0xc1,0x1d,0x9e,
29     0xe1,0xf8,0x98,0x11,0x69,0xd9,0x8e,0x94,0x9b,0x1e,0x87,0xe9,0xce,0x55,0x28,0xdf,
30     0x8c,0xa1,0x89,0x0d,0xbf,0xe6,0x42,0x68,0x41,0x99,0x2d,0x0f,0xb0,0x54,0xbb,0x16
31 };
32
33 void SubBytes(uint8_t* state) {
34     for (int i = 0; i < 16; i++)
35         state[i] = pgm_read_byte(&sbox[state[i]]);
36 }
37
38 void ShiftRows(uint8_t* state) {
39     uint8_t temp;
40
41     temp = state[1];
42     state[1] = state[5];
43     state[5] = state[9];
44     state[9] = state[13];
45     state[13] = temp;
46
47     temp = state[2];
48     state[2] = state[10];
49     state[10] = temp;
50
51     temp = state[6];
52     state[6] = state[14];
53     state[14] = temp;
54
55     temp = state[3];
56     state[3] = state[15];
57     state[15] = state[11];
58     state[11] = state[7];
59     state[7] = temp;
60 }
61
62 void MixColumns(uint8_t* s) {
63     for (int c = 0; c < 4; c++) {
64         int i = c * 4;
65         uint8_t a = s[i];
66         uint8_t b = s[i+1];
67         uint8_t c_ = s[i+2];
68         uint8_t d = s[i+3];
69
70         uint8_t e = a ^ b ^ c_ ^ d;
71         s[i] ^= e ^ ((a ^ b) << 1);
72         s[i + 1] ^= e ^ ((b ^ c_) << 1);
73         s[i + 2] ^= e ^ ((c_ ^ d) << 1);
74         s[i + 3] ^= e ^ ((d ^ a) << 1);
75     }
76 }
77
78 void AddRoundKey(uint8_t* state, const uint8_t* roundKey) {
79     for (int i = 0; i < 16; i++)
80         state[i] ^= roundKey[i];
81 }
82
83 // ===== AES Key Schedule (simple fixed key) =====
84 uint8_t key[16] = {
85     0x2b,0x7e,0x15,0x16,
86     0x28,0xae,0xd2,0xa6,
87     0xab,0xf7,0x15,0x88,
88     0x09,0xcf,0x4f,0x3c
89 };
90
91 void AES128_Encrypt(uint8_t* state) {
92     AddRoundKey(state, key);
93     for (uint8_t round = 1; round <= 9; round++) {
94         SubBytes(state);
95         ShiftRows(state);
96         MixColumns(state);
97         AddRoundKey(state, key);
98     }
99     SubBytes(state);
100    ShiftRows(state);
101    AddRoundKey(state, key);
102 }
103
104 const int triggerPin = 8;
105 uint8_t plaintext[16] = {
106     0x00,0x11,0x22,0x33,
107     0x44,0x55,0x66,0x77,
108     0x88,0x99,0xaa,0xbb,
109     0xcc,0xdd,0xee,0xff
110 };
111
112 void setup() {
113     pinMode(triggerPin, OUTPUT);
114     digitalWrite(triggerPin, LOW);
115 }
```

How the Arduino Passkey Lock Works

- THE ARDUINO UNO R3 ACTS LIKE A REAL DIGITAL LOCK.
- IT SAVES A SECRET 4-DIGIT KEY INSIDE THE CODE.
- IT LISTENS FOR NUMBERS FROM THE USER.
- WHEN 4 NUMBERS ARE ENTERED, IT CHECKS THE KEY.
- THE CODE ALSO ADDS A SMALL DELAY TO IMITATE REAL CPU WORK

ACTION

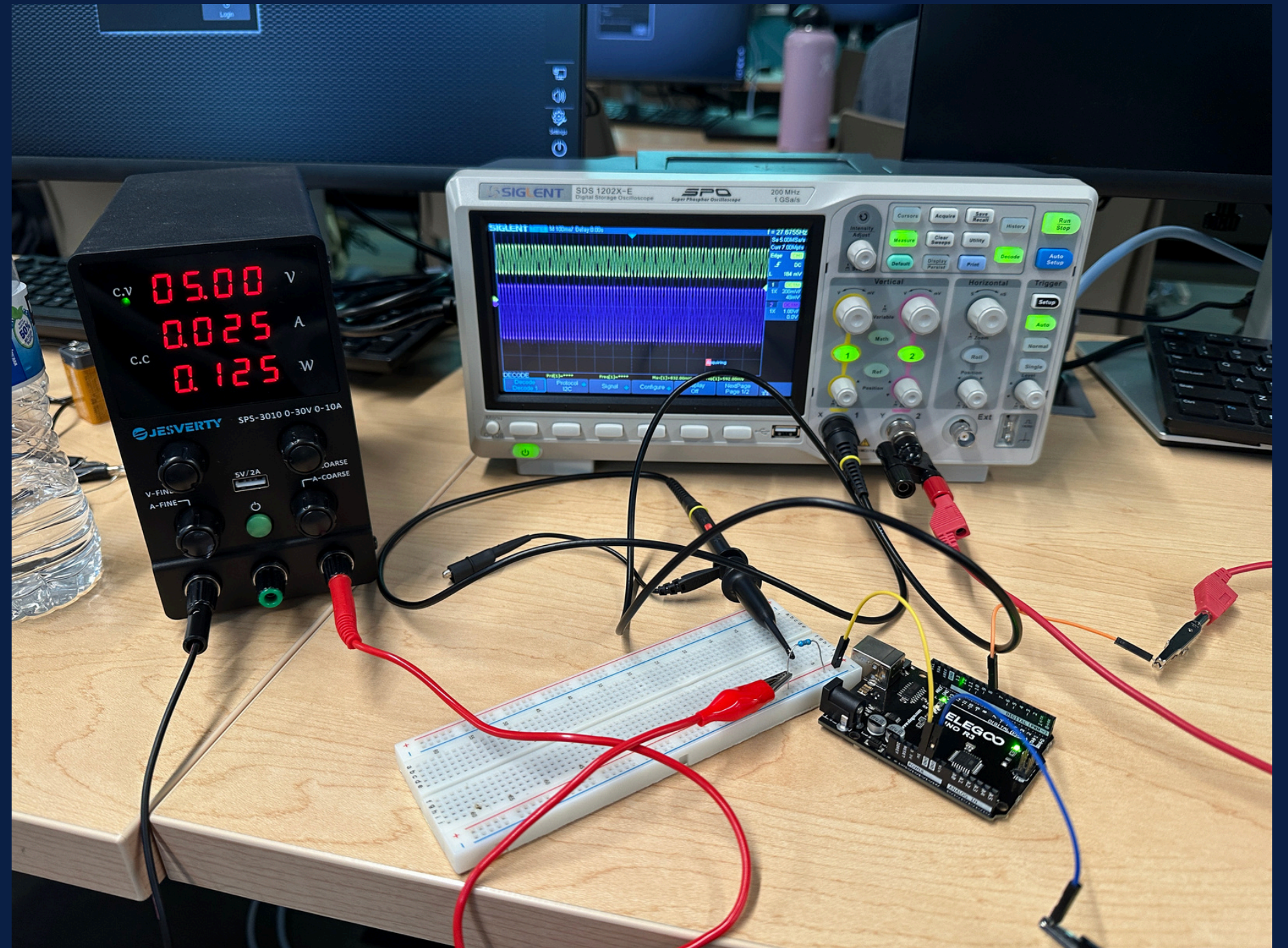
Power Analysis attacks



Hardware Setup

Overview of connections using shunt resistor and oscilloscope setup

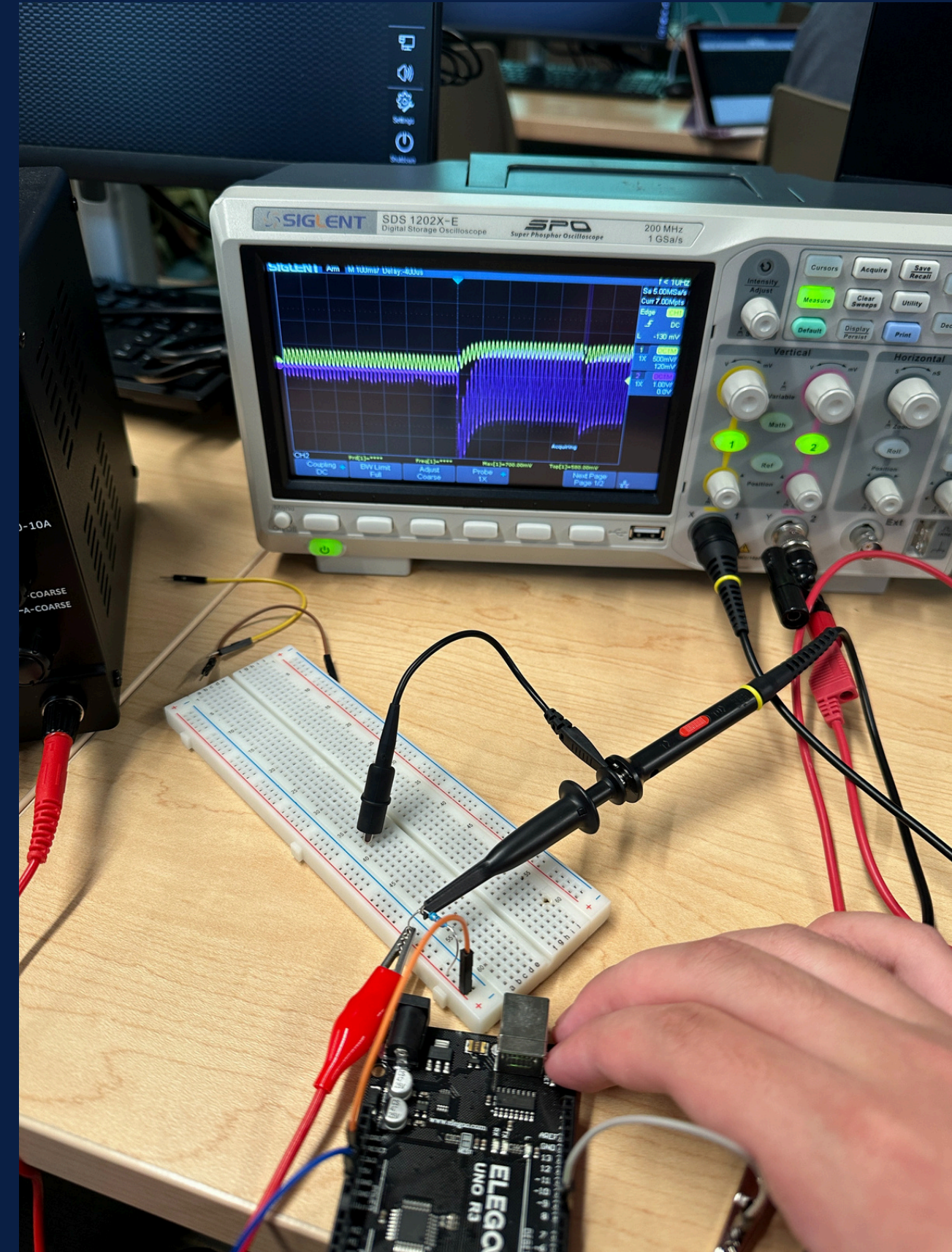
- Shunt resistor connected to GND
- Oscilloscope probes for waveform capture
- Trigger pin for synchronization



How the Arduino Passkey Lock Works Internally

Each digit triggers different CPU operations, which creates a unique power signature.

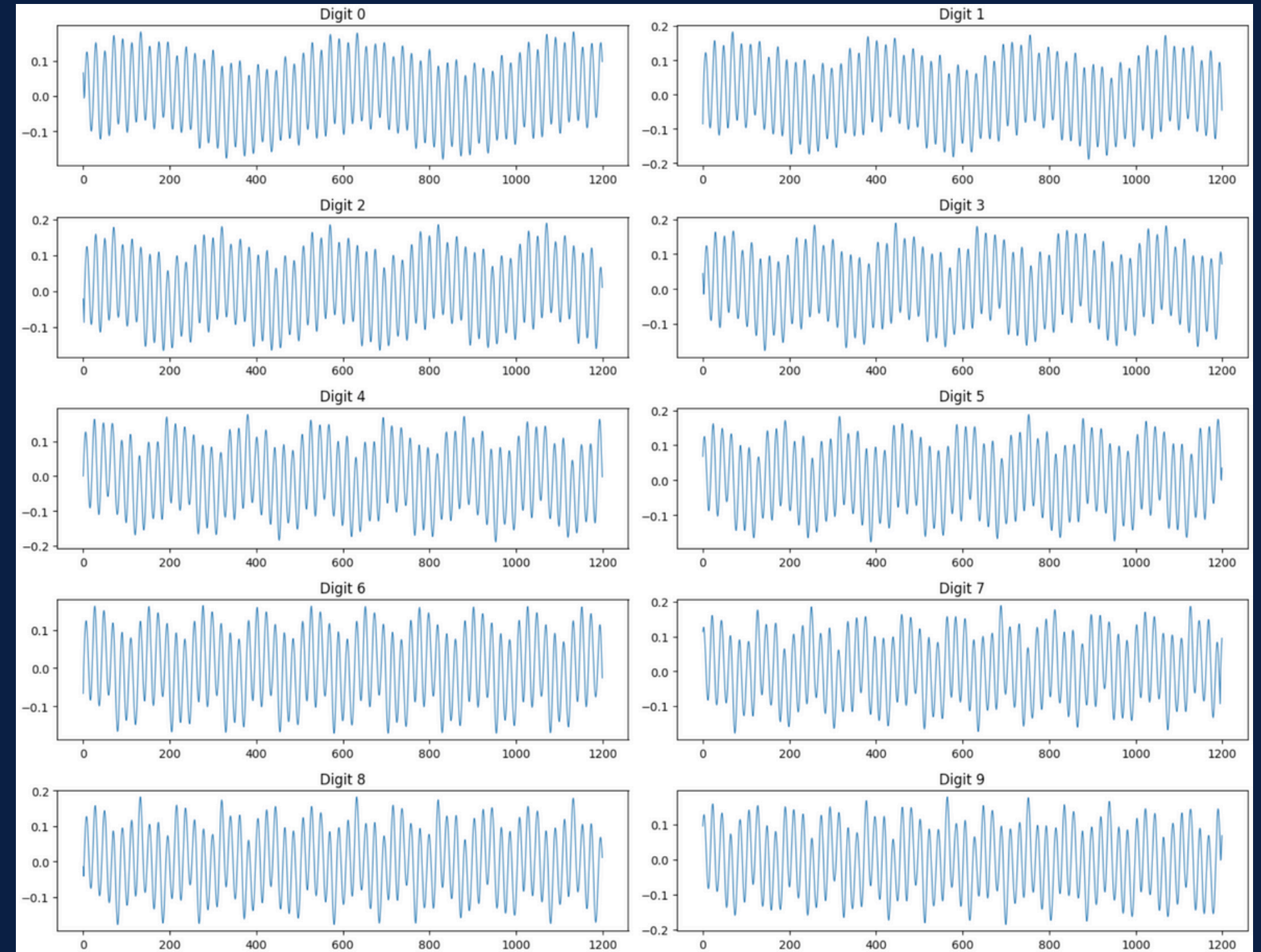
- The Arduino UNO R3 runs a simple encryption-like check to validate a 4-digit passkey.
- Each input digit is processed by code that performs arithmetic and lookup operations.
- Different digits cause different CPU operations → different power usage.
- These small differences create unique power waveforms on the oscilloscope.



Capturing the Power Signals

In order to clearly show how a side-channel attack works, I used Python to interact with my Arduino board

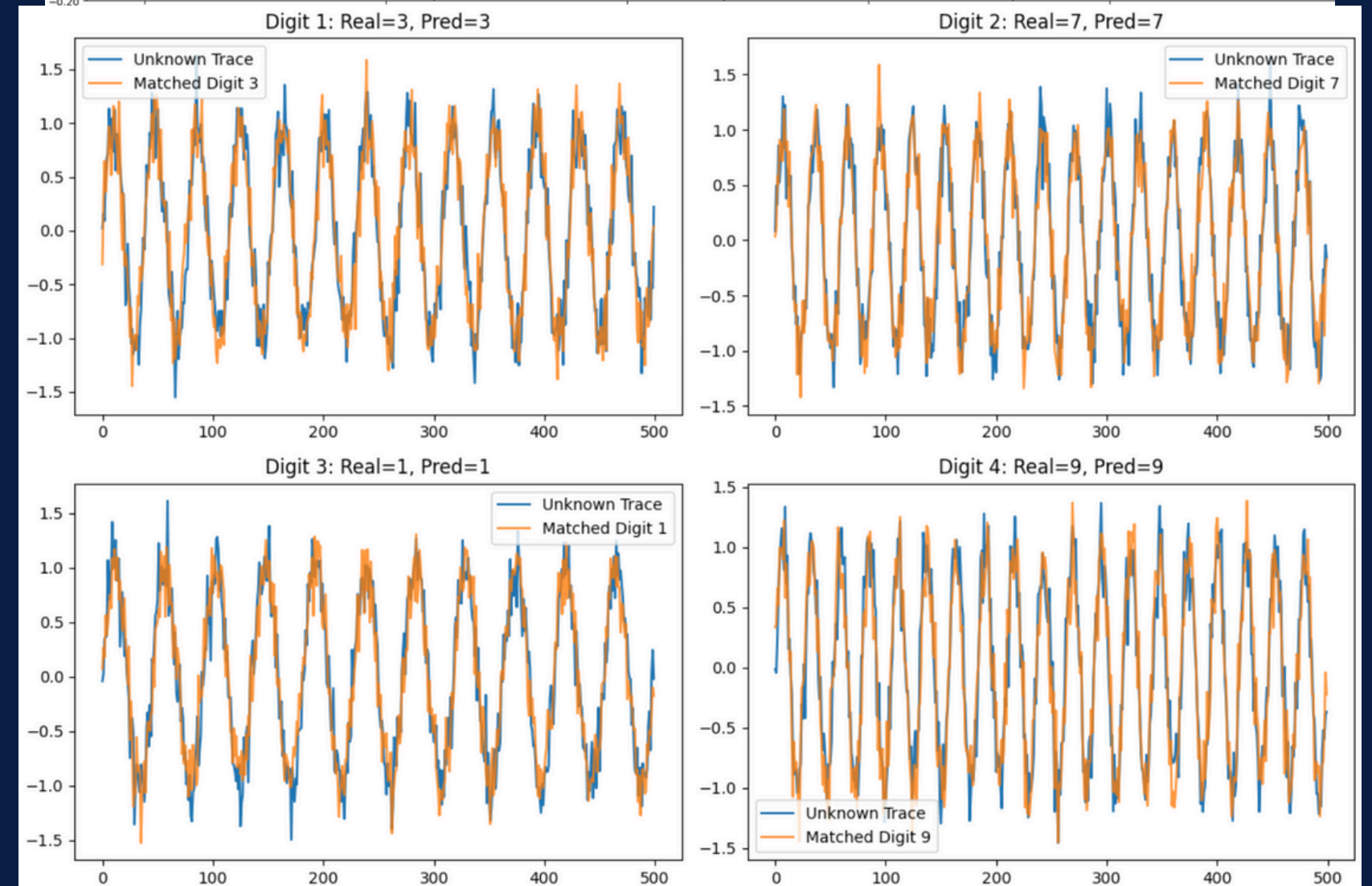
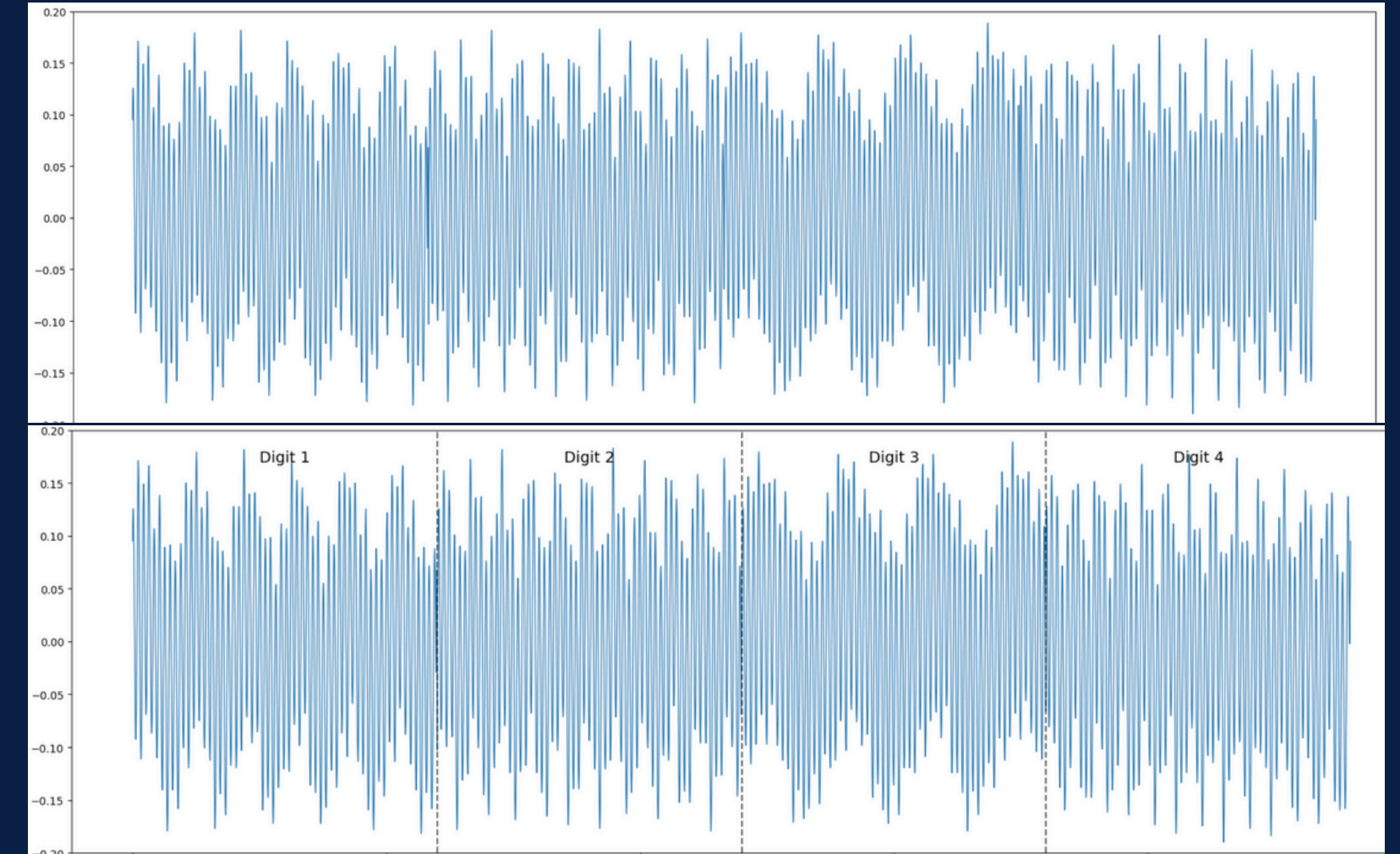
- PySerial is a Python library that enables access to the serial port.
- To observe how the Arduino processes each digit of the passkey.
- Each digit triggers slightly different CPU instructions.
- These internal differences change the Arduino's power consumption.



Recovering the 4-Digit Passkey

Automatic Python script

- Python compares each new trace with the learned digit fingerprints.
- The system identifies the digit based only on the waveform shape.
- The Arduino's secret PIN: 3 → 7 → 1 → 9



RESULTS

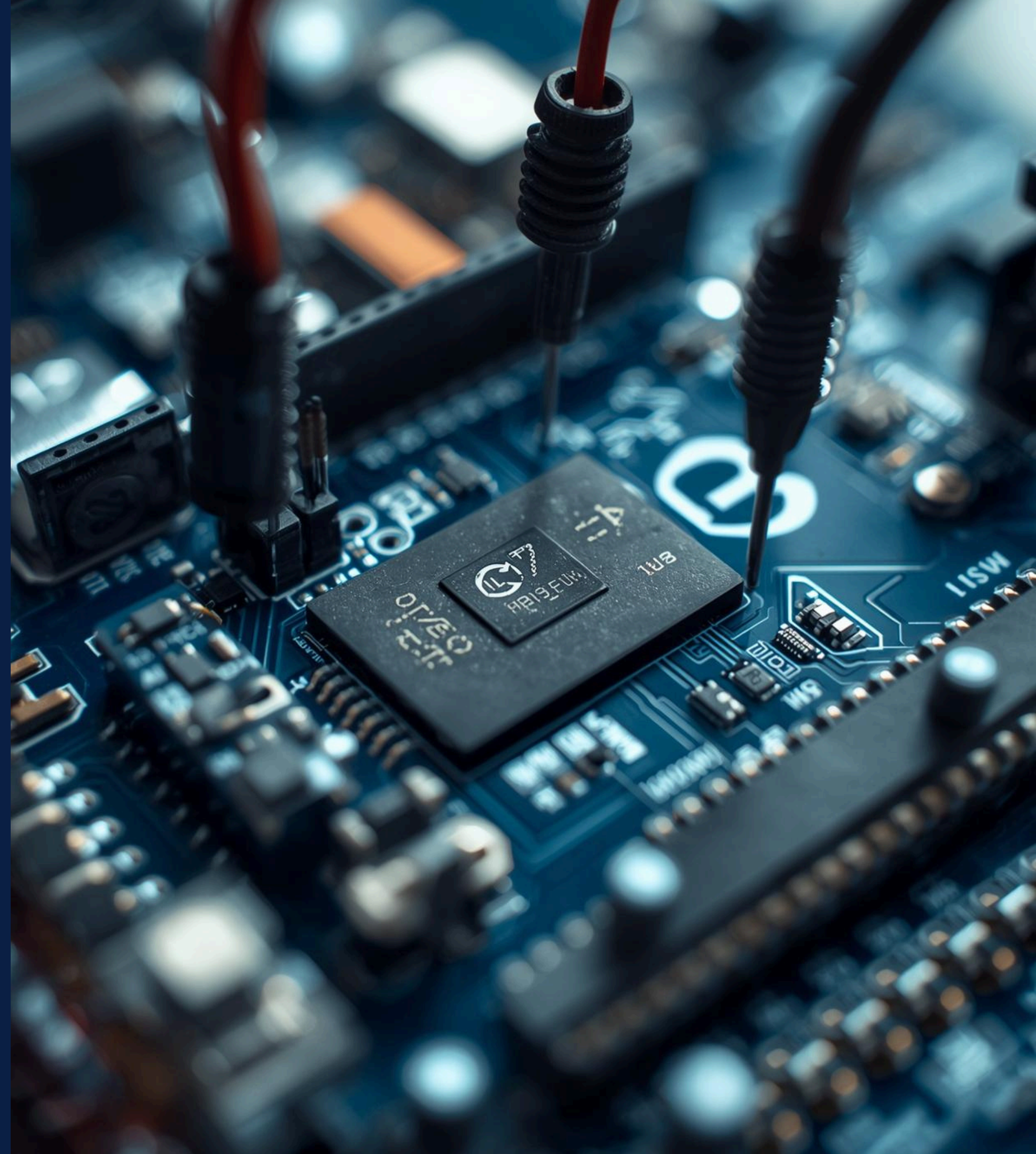
**Success... but
caught ! ! !**

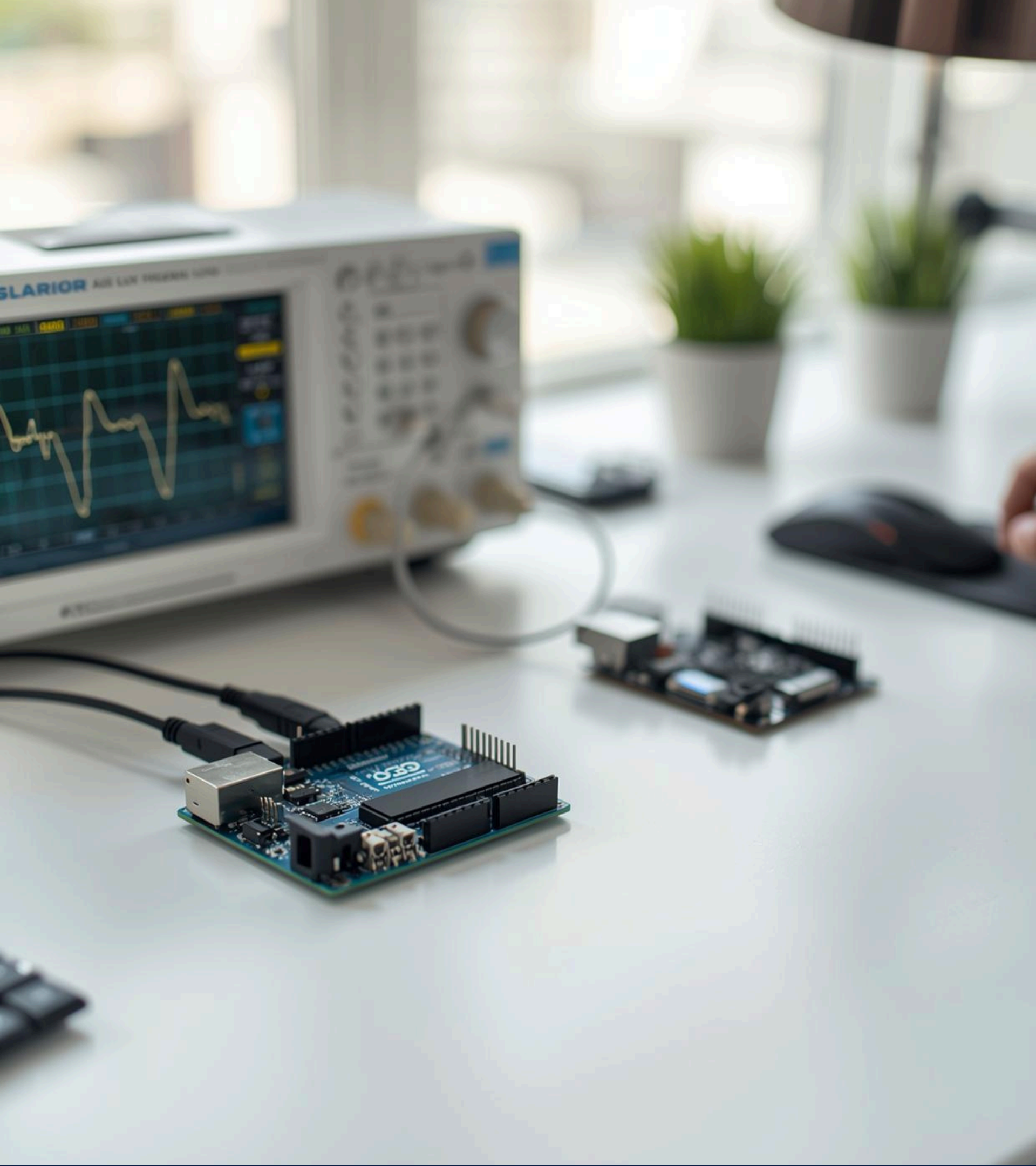
I SAW YOU ON THE CAMERA!!! — VINCENT



Vincent's Hardware Security Upgrade

1. ADD NOISE TO POWER CONSUMPTION
2. ADD HARDWARE POWER FILTERING
3. ADD PIN RETRY LIMITS & LOCKOUTS





Thank You !